Auth & Bring-up Setup — Windows

Steps and commands only. Everything runs in **PowerShell 7** (`pwsh`) from the repo root. `<like-this>` = placeholder.

> This is the Windows edition of `keycloak/SETUP.md`. Every `.sh` script in this repo has a `.ps1` twin that does the same job through the Keycloak Admin REST API and .NET crypto — no `jq`, `openssl`, `python3`, `fold` or `base64` needed. Keys and PEMs are written in identical formats, so a client created on Windows works on Linux and vice versa.

---

0. Prereqs

0a. Install the tools

Run in an **Administrator** PowerShell. `winget` ships with Windows 11.

```
winget install --id Git.Git              -e --source winget   # Git for Windows
winget install --id Docker.DockerDesktop -e --source winget   # Docker Desktop (WSL2 backend)
winget install --id Microsoft.PowerShell -e --source winget   # PowerShell 7 (pwsh)
```

Optional, only for the extras noted later:

```
winget install --id OpenJS.NodeJS.LTS    -e --source winget   # for keycloak/token-server.mjs (§8)
winget install --id Python.Python.3.12   -e --source winget   # for the CDRS dump script (§9a)
winget install --id GitHub.cli           -e --source winget   # easiest GitHub auth (§0b)
```

Then:

1. Launch **Docker Desktop** once and let it finish starting (whale icon steady in the tray). Settings → General → **Use the WSL 2 based engine** must be on. Settings → Resources → WSL Integration → enable your distro if you use one.
2. Close the terminal and reopen **PowerShell 7** (`pwsh`), not Windows PowerShell 5.1 — the `.ps1` scripts declare `#Requires -Version 7.1`.
3. Verify:

```
git --version
docker version
docker compose version
$PSVersionTable.PSVersion        # 7.1 or newer
```

Not needed on Windows: `jq`, `openssl`, `wg` / `wg-quick`. `curl.exe` is already part of Windows 10/11 — in `pwsh` the bare name `curl` resolves to it, but the examples below spell out `curl.exe` so they also work in Windows PowerShell.

0b. GitHub access

The repo is private under the **ETech-Nepal** organisation. Before cloning:

1. Ask an ETech-Nepal org owner to add your GitHub account as a **collaborator with at least Read access** on `ETech-Nepal/openhim`.

2. Accept the invitation email (or https://github.com/ETech-Nepal/openhim/invitations).
3. Authenticate git. Easiest is the GitHub CLI:

```
gh auth login          # GitHub.com -> HTTPS -> login with a browser
```

Alternatives: a **Personal Access Token** (classic, `repo` scope) used as the password at the git prompt — Git Credential Manager caches it in Windows Credential Manager — or an SSH key (`ssh-keygen -t ed25519` , add the `.pub` to GitHub, then clone the `git@github.com:` URL).

4. Confirm access **before** cloning:

```
git ls-remote https://github.com/ETech-Nepal/openhim.git | Select-Object -First 1
```

`remote: Repository not found` here means the invite is not accepted or the credential is wrong — fix that first, it is not a clone problem.

0c. Clone the repo

Repository: https://github.com/ETech-Nepal/openhim

```
cd C:\GitHub                                          # or wherever you keep code
git clone https://github.com/ETech-Nepal/openhim.git
cd openhim
```

Every command from here on runs from this directory (the repo root).

> Keep the path short and free of spaces. Docker Desktop bind-mounts several paths from this tree (`openhim-config` , `hapi-config` , `jempi-config`); a path with spaces or a OneDrive-synced folder causes mount failures. Git for Windows converts line endings on checkout. The `.ps1` scripts do not care, but if you ever run the `.sh` twins through WSL and get `bad interpreter: /bin/bash^M` , re-clone with `git clone --config core.autocrlf=input` .

---

1. Environment files

Four services read a `.env` that is **not** in git. Create each from its example:

```
Copy-Item mediator\.env.example                mediator\.env
Copy-Item admin-mediator\.env.example          admin-mediator\.env
Copy-Item patient-portal-mediator\.env.example patient-portal-mediator\.env
Copy-Item fhir-intake-mediator\.env.example    fhir-intake-mediator\.env
```

The defaults are correct for a local Docker run. Only these need your input:

File	Variable	Set it to
all four	`OPENHIM_PASSWORD`	the OpenHIM root password from §3b — **blank in the examples**, mediators crash-loop until it is filled
`mediator\.env`	`JEMPI_BASE_URL`	`http://202.51.71.81:50000/JeMPI` (§4)
`admin-mediator\.env`	`JEMPI_BASE_URL`	`http://202.51.71.81:50000/JeMPI` (§4)
`patient-portal-mediator\.env`	`PORTAL_PATIENT_SHARED_SECRET`	only if you run the patient-portal BFF — must equal its `SESSION_JWT_SECRET`
`mediator\.env`	`FACILITY_REGISTRY_JSON`	only as the §7 fallback

Everything else — `OPENHIM_API_URL`, `HAPI_FHIR_BASE_URL`, `MEDIATOR_ROUTE_HOST`, the `OPENHIM_CHANNEL_ROLE` values, the admin-mediator scoring thresholds — is Docker-internal and should stay as shipped.

Roles that matter later: `mediator` and `fhir-intake-mediator` both bind to `np-fhir-clients`, which is the role `onboard-hospital.ps1` grants (§7).

> `.env` is in `.gitignore`. Never commit one — they hold the OpenHIM root password.

To set the password in all four without opening an editor:

```
foreach ($d in 'mediator','admin-mediator','patient-portal-mediator','fhir-intake-mediator') {
  (Get-Content "$d\.env") -replace '^OPENHIM_PASSWORD=.*', 'OPENHIM_PASSWORD=<your-password>' |
    Set-Content "$d\.env"
}
```

---

2. Containers

```
docker compose up -d
docker compose ps
```

Running: `hapi-db`, `mongo-db`, `openhim-core`, `openhim-console`, `fhir`, `keycloak`, `mediator`, `admin-mediator`, `patient-portal-mediator`, `fhir-intake-mediator`.

> **Never `docker compose down`** (wipes Keycloak's H2 store and with it the realm). Use `docker compose stop` / `docker compose start`. Mediators crash-loop until steps 3 and 6 are done — that is expected.

The first run pulls several GB of images. If a container exits immediately, read its log before anything else:

```
docker compose logs --tail 50 mediator
```

---

3. OpenHIM first login

3a. Accept the self-signed cert

1. Open https://localhost:8080/heartbeat in Edge/Chrome, click **Advanced** → **Continue**. Without this the Console's browser calls fail silently.

2. From the CLI you need `-k`: `curl.exe -k https://localhost:8080/heartbeat` (PowerShell-native: `Invoke-RestMethod https://localhost:8080/heartbeat -SkipCertificateCheck`).

3b. Root password

1. Console http://localhost:9000, log in `root@openhim.org` / `openhim-password`.
2. Set a new password. **Record it** — §3c needs it in four files.

3c. Propagate to mediators

Set `OPENHIM_PASSWORD` in `mediator\.env`, `admin-mediator\.env`, `patient-portal-mediator\.env`, `fhir-intake-mediator\.env` (§1), then:

```
docker compose up -d --force-recreate mediator admin-mediator patient-portal-mediator fhir-intake-mediator
```

> `docker compose restart` does **not** reload `.env` — use `up -d --force-recreate`.

Console → **Mediators**: all four green.

---

4. JeMPI connectivity — WireGuard is no longer required

JeMPI runs on the VPS `202.51.71.81`. **Port `50000` is now forwarded on that host**, so the API is reachable straight over the public internet — no tunnel, nothing extra to install on Windows:

```
curl.exe http://202.51.71.81:50000/JeMPI/countRecords
# {"goldenRecords":85,"interactions":85}
```

Point both mediators at it (§1):

```
JEMPI_BASE_URL=http://202.51.71.81:50000/JeMPI
```

in `mediator\.env` **and** `admin-mediator\.env`, then:

```
docker compose up -d --force-recreate mediator admin-mediator
docker exec mediator wget -qO- http://202.51.71.81:50000/JeMPI/countRecords   # from inside the container
```

Both the host check and the container check must return the JSON. A container reaches a public IP through Docker's NAT with no extra rules — the Linux `iptables MASQUERADE` workaround from the tunnel era does not apply.

> `:50000` is the open API (no Keycloak in front). `:50001` is the Keycloak-protected port; production will move to it later. Because `:50000` is now publicly exposed rather than tunnel-only, treat it as untrusted transport: restrict it to known source IPs on the JeMPI VPS firewall.

4a. WireGuard fallback (kept for reference)

Still valid if the port forward is withdrawn, or if you must reach `:50001` or other VPS-internal services. The tunnel address for JeMPI was `10.8.0.1`.

- **Windows:** install the official client (`winget install --id WireGuard.WireGuard -e`), then **Add Tunnel** → **Import from file** with the `wg0.conf` you were issued and **Activate**. Ensure the peer's `AllowedIPs` includes `10.8.0.0/24`. Docker Desktop containers route through the Windows host, so no extra rules are needed; if a container still cannot reach `10.8.0.1`, restart Docker Desktop after activating the tunnel so the WSL network picks up the route.
- **Linux host / VPS:**

```
sudo wg-quick up wg0                                          # config at /etc/wireguard/wg0.conf
curl http://10.8.0.1:50000/JeMPI/countRecords                 # from host
docker exec mediator wget -qO- http://10.8.0.1:50000/JeMPI/countRecords   # from a container

# host works but container hangs:
sudo iptables -t nat -A POSTROUTING -s 172.16.0.0/12 -o wg0 -j MASQUERADE
sudo sysctl -w net.ipv4.ip_forward=1
```

If you go back to the tunnel, set `JEMPI_BASE_URL=http://10.8.0.1:50000/JeMPI` and force-recreate the two mediators again.

---

5. Keycloak realm

Admin Console http://localhost:8083, bootstrap admin `admin` / `admin`.

```
(Invoke-RestMethod http://localhost:8083/realms/openhie/.well-known/openid-configuration).issuer
```

If it prints `http://localhost:8083/realms/openhie`, the realm exists — go to §7. Otherwise:

```
pwsh -File .\keycloak\setup.ps1
```

(or by hand: realm dropdown → **Create realm** → name `openhie` → **Create**).

`setup.ps1` also creates `demo-client` + user `demo` / `demo123` and prints the demo user's `sub`. A password-grant demo token, no `jq` needed:

```
(Invoke-RestMethod -Method Post -Uri 'http://localhost:8083/realms/openhie/protocol/openid-connect/token'
`
   -Body @{ grant_type='password'; client_id='demo-client'; username='demo'; password='demo123'
}).access_token
```

---

6. Realm key into OpenHIM

```
pwsh -File .\keycloak\extract-realm-pubkey.ps1
docker compose up -d --force-recreate openhim-core
docker compose restart mediator admin-mediator patient-portal-mediator fhir-intake-mediator
```

> A realm rebuild rotates Keycloak's RSA key → re-run this or every token fails `JWT could not be verified`.

OpenHIM env (already in `docker-compose.yml`): `authentication_jwt_algorithms` = `RS256` (plain string), `authentication_jwt_audience` = `account`, `authentication_jwt_issuer` = `http://localhost:8083/realms/openhie`.

Transaction-port errors:

Code	Meaning
401	token missing / bad signature / issuer or audience mismatch
403	Client mapped, but lacks a channel role
404	no channel matched the URL

> **Windows gotcha.** If Docker Desktop started the stack before `realm-public.pem` existed, it created an empty **directory** at `openhim-config\realm-public.pem`. `extract-realm-pubkey.ps1` detects this and prints the fix: stop `openhim-core`, `Remove-Item -Recurse openhim-config\realm-public.pem`, re-run the script, force-recreate.

---

7. Per-hospital client

One command per hospital — NHFR code + name. Creates the Organization, keypair, Keycloak client + `sub`-override mapper, and OpenHIM Client; verifies the token `sub`:

```
pwsh -File .\keycloak\onboard-hospital.ps1 3130400221 "Bharatpur Hospital"
```

Idempotent. Output gives the private-key path (`keycloak\keys\<nhfr>.key.pem`) — hand it to the hospital over a secure channel, then delete your copy. `keycloak/keys/` is gitignored.

System	Value
FHIR	`Organization.identifier` = `<nhfr-id>`; `Organization.id` = server UUID
Keycloak	client `<nhfr-id>`, `private_key_jwt`, token `sub` = `<nhfr-id>`
OpenHIM	Client **ID** = `<nhfr-id>`, role `np-fhir-clients`
Mediator	`X-OpenHIM-ClientID` = `<nhfr-id>` → `GET Organization?identifier=nhfr|<nhfr-id>`

Prereqs: §2, §3c, §6.

Just an Organization, no client: `pwsh -File .\keycloak\create-org.ps1 <nhfr-id> "<name>"`.

> NHFR system string: deployed stack uses `http://mohp.gov.np/fhir/NamingSystem/nhfr` (NP Core's canonical is `https://nhfr.mohp.gov.np`).
> `sub` override rejected → `onboard-hospital.ps1` fails at 4/5. Fallback: OpenHIM `clientID` stays the Keycloak UUID; add `FACILITY_REGISTRY_JSON={"<uuid>":"<nhfr-id>"}` to `mediator\.env` + `docker compose up -d --force-recreate mediator`.

Script overrides via environment: `KEYCLOAK_URL` (default `http://localhost:8083`), `KEYCLOAK_ADMIN_USER` / `KEYCLOAK_ADMIN_PASSWORD` (default `admin` / `admin`), plus `FHIR_BASE_URL` and `ACCESS_TOKEN` for `create-org.ps1`.

---

8. Smoke test

```
$TOKEN = & .\keycloak\get-jwt-client-token.ps1 3130400221

$body = @'
{"resourceType":"Patient","name":[{"family":"SHRESTHA","given":["KAMANA"]}],
 "gender":"female","birthDate":"2001-06-25",
 "telecom":[{"system":"phone","value":"9800000000"}]}
'@

Invoke-RestMethod -Method Post -Uri 'https://localhost:5000/fhir/Patient' `
  -SkipCertificateCheck `
  -Headers @{ Authorization = "Bearer $TOKEN" } `
  -ContentType 'application/fhir+json' -Body $body
```

`201` with a UUID `id` and `nepal-health-id` in `identifier` = the full chain works. Console → **Transactions** → the request → **orchestrations**.

> **Reading a failure.** On an error OpenHIM echoes **your request body back** as the response body, and `Invoke-RestMethod` surfaces that as the exception message — so a failed smoke test looks like it printed your own Patient JSON back at you. That text tells you nothing; you need the status code and the core log. Wrap the call to get both:
> ``powershell try { Invoke-WebRequest -Method Post -Uri 'https://localhost:5000/fhir/Patient' -SkipCertificateCheck -Headers @{ Authorization = "Bearer $TOKEN" } ` -ContentType 'application/fhir+json' -Body $body } catch { "STATUS: $([int]$*.Exception.Response.StatusCode)" $*.ErrorDetails.Message }
> docker compose logs --tail 40 openhim-core ```
> Then read the status against the §6 table (401 auth / 403 role / 404 channel). Note `Invoke-WebRequest`'s `.Content` comes back as a **byte array** for `application/fhir+json` — use `Invoke-RestMethod`, which parses it into an object, once the call succeeds.

Postman: `node keycloak\token-server.mjs`, then http://localhost:4099/token/3130400221.

---

9. FHIR Intake mediator (`/ingest`)

`POST /ingest` in front of the orchestrator:

`Content-Type`	Body	Action
`application/json`	one raw CDRS encounter row	validate → Patient + Encounter Bundle → forward
`application/fhir+json`	resource or `transaction` / `batch` Bundle	forward unmutated
other	—	`415`

Channel `NP FHIR Intake`, `^/ingest/?$`, `POST`, route `fhir-intake-mediator:4003`, role `OPENHIM_CHANNEL_ROLE` in `fhir-intake-mediator\.env` (`np-fhir-clients` to reuse §7 clients).

Raw-row behaviour: `403` if no `X-OpenHIM-ClientID`; `GET Organization?identifier=nhfr|<nhfr-id>` → `422` if 0 or >1 match; stamps `managingOrganization` / `serviceProvider` = the resolved `Organization/<uuid>`. Inbound `X-NHFR-ID` / `organization_id` ignored.

9a. Dump script

Needs Python 3 (§0a) and the hospital's private key from §7.

```
cd FHIR_mediator
Copy-Item .env.example .env     # KEYCLOAK_CLIENT_ID=<nhfr-id>, PRIVATE_KEY_FILE, SQLITE_PATH

$env:DRY_RUN = '1'; python main.py    # preview
Remove-Item Env:\DRY_RUN
python main.py                        # raw rows
python main.py --FHIR                 # client-mapped Bundles
```

`PRIVATE_KEY_FILE` takes a Windows path — use forward slashes or doubled backslashes (`C:/keys/3130400221.key.pem`). Progress is bookmarked per `encounter_number`, so a re-run resumes rather than re-sends.

9b. Smoke test

```
$TOKEN = & .\keycloak\get-jwt-client-token.ps1 3130400221

$row = @'
{"hospital_patient_number":"2405078","encounter_number":"SMOKE-1",
 "first_name":"CHANDRIKA","last_name":"KANDEL","sex":"Female",
 "department_type":"C","department_id":"22","department_name":"Family Medicine",
 "visit_date_ad_time":"2026-07-28 00:02:00.000"}
'@

Invoke-RestMethod -Method Post -Uri 'https://localhost:5000/ingest' `
  -SkipCertificateCheck `
  -Headers @{ Authorization = "Bearer $TOKEN" } `
  -ContentType 'application/json' -Body $row
```

`201`. A second POST of the same `encounter_number` must not create a duplicate.

---

10. Deploy on a VPS

Steps 1–9 assume everything is on `localhost`. On a VPS the token issuer must match across Keycloak, OpenHIM, and every client, or all tokens 401. Let `<vps>` = the VPS public host or domain.

10a. Keycloak reachable + stable issuer

`docker-compose.yml`, `keycloak` service:

- Port: `127.0.0.1:8083:8080` → `8083:8080`.
- Add env: `KC_HOSTNAME: https://<vps>:8083` (or the proxy URL from 10d).
- `start-dev` is fine for testing. For production also set `KC_DB: postgres` + DB env and drop `--import-realm`; `start-dev`'s H2 store is lost on any `docker compose down`.

10b. Point OpenHIM + clients at that issuer

- `docker-compose.yml`, `openhim-core`: `authentication_jwt_issuer: "https://<vps>:8083/realms/openhie"`.

- `FHIR_mediator/.env` (each hospital box): `KEYCLOAK_URL=https://<vps>:8083`.
- `iss` in a minted token = Keycloak's `KC_HOSTNAME`. All three must be byte-identical.

```
docker compose up -d --force-recreate keycloak openhim-core
pwsh -File .\keycloak\extract-realm-pubkey.ps1
docker compose up -d --force-recreate openhim-core
docker compose restart mediator admin-mediator patient-portal-mediator fhir-intake-mediator
```

10c. Firewall

Expose only:

Port	Service
5000	OpenHIM secure transaction port (client traffic)
8083	Keycloak
9000	OpenHIM Console — VPN / admin IP only

Keep internal (no host rule): `8080` OpenHIM API, `8081` HAPI, `5433` hapi-db, `27017` mongo.

JeMPI `50000` is deliberately published on the JeMPI VPS (§4) — restrict it to the OpenHIM host's IP rather than leaving it open to the world.

10d. TLS

- Testing: `:5000` is already TLS with a self-signed cert. Clients keep `curl.exe -k` / `Invoke-RestMethod -SkipCertificateCheck` / `OPENHIM_TRUST_SELF_SIGNED=true`.
- Production: put Caddy or nginx with a real cert in front of `:5000` and `:8083`; set `KC_HOSTNAME` / `authentication_jwt_issuer` to the proxy URL.

10e. Secrets

Change before exposing the host:

- Keycloak `KC_BOOTSTRAP_ADMIN_PASSWORD` (was `admin`).
- OpenHIM root password (§3b) → propagate to every `*/.env` (§3c).
- Production: `hapi-db` and `mongo-db` credentials too.

10f. JeMPI from the VPS

With the `:50000` port forward in place (§4) there is nothing to set up — the VPS reaches `http://202.51.71.81:50000/JeMPI` directly. Only if you fall back to the tunnel do you need §4a on the VPS: `wg-quick up wg0`, `ip_forward`, and the `MASQUERADE` rule, or the mediators cannot reach `10.8.0.1:50000`.

10g. Never `docker compose down`

Same rule as local — wipes Keycloak's H2 realm. Use `stop` / `start`. A VPS you depend on should be on `KC_DB=postgres` (10a) so this stops being fragile.

Quick reference

Task	Command (PowerShell 7)
Clone	`git clone https://github.com/ETech-Nepal/openhim.git`
Create the `.env` files	`Copy-Item <svc>\.env.example <svc>\.env` (§1)
Bring the stack up	`docker compose up -d`
Reload a changed `.env`	`docker compose up -d --force-recreate <svc>`
Create the realm	`pwsh -File .\keycloak\setup.ps1`
Create/update an Organization	`pwsh -File .\keycloak\create-org.ps1 <nhfr-id> "<name>"`
Re-extract realm key	`pwsh -File .\keycloak\extract-realm-pubkey.ps1`
Onboard a hospital	`pwsh -File .\keycloak\onboard-hospital.ps1 <nhfr-id> "<name>"`
Hospital token	`.\keycloak\get-jwt-client-token.ps1 <nhfr-id>`
Postman token helper	`node keycloak\token-server.mjs` (`:4099`)
Check JeMPI	`curl.exe http://202.51.71.81:50000/JeMPI/countRecords`
JeMPI tunnel (only if needed)	WireGuard client → Activate `wg0` (§4a)
Push data	`cd FHIR_mediator; python main.py` (`--FHIR` for bundles)

`keycloak\_common.ps1` holds the shared helpers (admin login, REST wrapper, PEM/base64url, JWT claim reader) and is dot-sourced by the others — never run it directly.

Troubleshooting

Symptom	Fix
`Repository not found` on clone	GitHub invite not accepted / wrong credential (§0b)
`.ps1` won't run — *execution of scripts is disabled*	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, or invoke as `pwsh -File .\script.ps1`
`#Requires -Version 7.1` error	you are in Windows PowerShell 5.1 — open `pwsh`
`docker: error during connect`	Docker Desktop not running / WSL2 engine off (§0a)
Console calls fail, network error	accept the cert — https://localhost:8080/heartbeat (3a)
Mediator missing under **Mediators**	`OPENHIM_PASSWORD` ≠ root password (3c)
Mediator crash-loops on boot	`.env` missing or `OPENHIM_PASSWORD` blank (§1)
`.env` change ignored	used `restart`, not `up -d --force-recreate`
a failed call "returns" your own request JSON	that is OpenHIM's error body, not a diagnosis — get the status code + `docker compose logs openhim-core` (§8)
`JWT could not be verified: secret or public key must be provided`	OpenHIM has **no** key loaded — `realm-public.pem` is empty or a directory, see the next row
`realm-public.pem` is a directory	Docker created it before §6 ran — `docker compose stop openhim-core`, `Remove-Item -Recurse openhim-config\realm-public.pem`, re-run `extract-realm-pubkey.ps1`, force-recreate
`JWT could not be verified` (other)	`realm-public.pem` stale — re-run §6
401 at `:5000` fresh token	issuer mismatch — `iss` must be `http://localhost:8083/realms/openhie`
403 at `:5000`	Client lacks role — re-run `onboard-hospital.ps1` (§7)
404 at `:5000/ingest`	`NP FHIR Intake` channel missing — check `fhir-intake-mediator`
403 at `:5000/ingest`	set `OPENHIM_CHANNEL_ROLE=np-fhir-clients` on `fhir-intake-mediator`
422 at `:5000/ingest`	no Organization with NHFR identifier `<nhfr-id>` — `create-org.ps1` (§7)
`onboard-hospital.ps1` fails at 4/5	Keycloak won't override `sub` — `FACILITY_REGISTRY_JSON` fallback (§7)
`502` + `JeMPI register failed: timeout`	`JEMPI_BASE_URL` still `10.8.0.1` — switch to `202.51.71.81:50000` and force-recreate (§4)
Keycloak realm gone after a stop	`docker compose down` was used — use `stop` / `start`